



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Gym Buddy

Name: Stolniceanu Iustin

Group: 30237

Table of Contents

<i>Deliverable 1</i>	3
Project Specification	3
Functional Requirements	3
Use Case Model	3
Use Cases Identification:	3
UML Use Case Diagrams	4
Supplementary Specification	5
Non-functional Requirements.....	5
Design Constraints	5
Glossary	5
<i>Deliverable 2</i>	6
Domain Model	6
Architectural Design	6
Conceptual Architecture	6
Package Design.....	7
Component and Deployment Diagram	8
<i>Deliverable 3</i>	9
Design Model	9
Dynamic Behavior	9
Class Diagram	10
Data Model	11
<i>System Testing</i>	11
<i>Future Improvements</i>	12
<i>Conclusion</i>	12
<i>Bibliography</i>	12

Deliverable 1

Project Specification

GymBuddy is a full-stack web app designed to help fitness enthusiasts log and manage their workouts. Moving away from cumbersome spreadsheets, the system allows users to seamlessly track their gym progress. Users can create, modify, view and delete specific Gym Days, they can log individual exercises performed during those sessions, including the number of sets, repetitions, the weights used. The application will generate graphs and maybe predict with machine learning(work in progress, if I can do my Machine Learning project on this theme too) if a specific day is good or not for gym based on sleep, heart rate etc.

Functional Requirements

User & Account management

1. User authentication: the system must allow users to register for an account, log in and log out securely.
2. Role-based access control: the system must support two distinct user roles: Standard User and Administrator
3. Data Isolation: The app must ensure that standard users can only view, modify and delete their own personal workout logs and not other users' logs.

Administrator Capabilities

4. Exercise catalog management: The system must allow an administrator to create, update and delete predefined Exercise Types(e.g. Bench Press, Squat etc) that standard users can select from when logging a workout

Workout Management

5. Manage Gym days: the system must allow the user to create a new gym day entry, modify an existing one, view past gym dates and delete a gym day.
6. Log exercises: The system must allow the user to add specific exercises (chosen from the Admin's catalog or custom exercises defined by them) to a selected gym day.
7. Track Metrics: The system must allow the user to input sets, repetitions, and weight values for each logged exercise
8. Manage logged exercises: The system must allow the user to edit or remove a specific exercise entry from a gym day if a mistake was made.
9. The system must allow users to input physical measurements and communicate with a dedicated Python microservice to predict and log body fat percentage.

Analytics & History

10. View History: The system must display a chronological history of gym days and their associated exercises
11. Generate insights: The system must aggregate workout data to provide visual insights(maybe dynamic graphs + others), showing a user's progress and volume trends for specific exercises over time.
12. Export History: The system must allow users to export their workout into an XML Format using a Strategy-based export engine.

Use Case Model

Use Cases Identification:

Use-case: Register/Login

Level: User Goal

Primary actor: Regular User / administrator

Main success scenario:

1. User enters email and password
2. System validates credentials against the database
3. System identifies the user and returns their unique Profile ID.
4. User is redirected to their personal dashboard

Extensions: 1a. Invalid email format: system prompts for a valid email

2a. Incorrect password: System displays "invalid credentials".

Use Case: Manage Gym Days

Level: User Goal

Primary actor: Regular User

Main success scenario:

1. User selects "Add Gym Day"
2. User inputs date and optional notes (such as sleep, tiredness etc.)
3. User saves the entry
4. System adds the day to the chronological history list

Extension: User selects an existing day to Edit or Delete it.

Use Case: Track Body Metrics

Level: User Goal

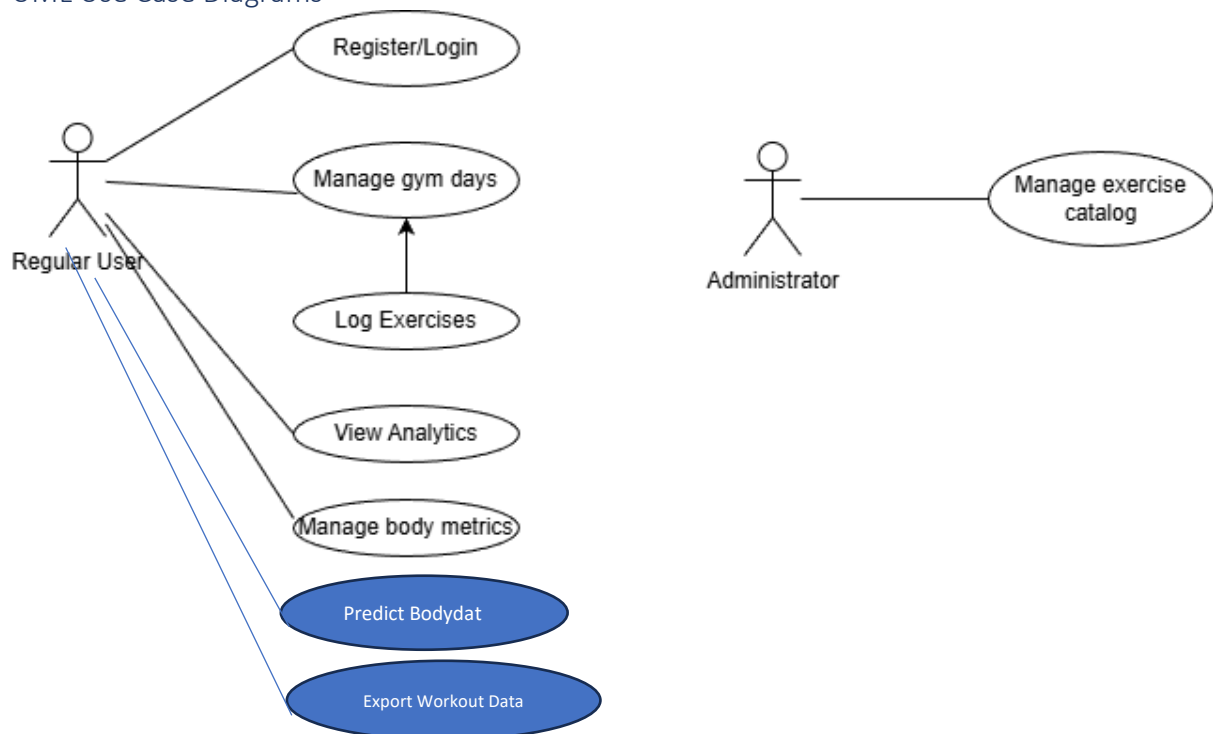
Primary actor: Regular User

Main success scenario:

1. User navigates to "body stats" section
2. User enters current weight
3. User enters hip, thigh, arm, waist circumferences.
4. System saves the entry with the current timestamp
5. System calculates BMI and shows weight change relative to the last entry
6. System activates the body weight graph
7. System predicts, on user's wish, the body fat of the user.

Extension: Out-of-range data: User enters "0" weight or a ridiculous amount => system triggers a validation error.

UML Use Case Diagrams



Supplementary Specification

Non-functional Requirements

1. **Security:** The system must securely store user credentials using industry-standard password hashing and secure the API endpoints using a custom identifier (X-User-Id header) to track the active session between the React frontend and the microservices. This is suitable for our system since we introduced a login system and role-based access control. We don't want to let anyone log into anyone else's account.
2. **Performance:** The backend REST API must process standard CRUD operations and return a response in under 500 milliseconds under normal load, as fitness users will likely log this data while actively working out at the Gym, so a fast, snappy response is critical.
3. **Maintainability:** The backend source code must strictly adhere to SOLID principles and utilize a clearly separated Layered Architecture (most likely Model, View, Controller). This is a core requirement because it ensures the application is highly decoupled, so we can add features over time and not build the app like a monolith.
4. **Usability:** While built as a web application using React, the frontend UI must be fully responsive and optimized for mobile screen sizes, as the primary environment for this application is the gym. Users will almost exclusively interact with the client application via their smartphones, making mobile usability a top priority.

Design Constraints

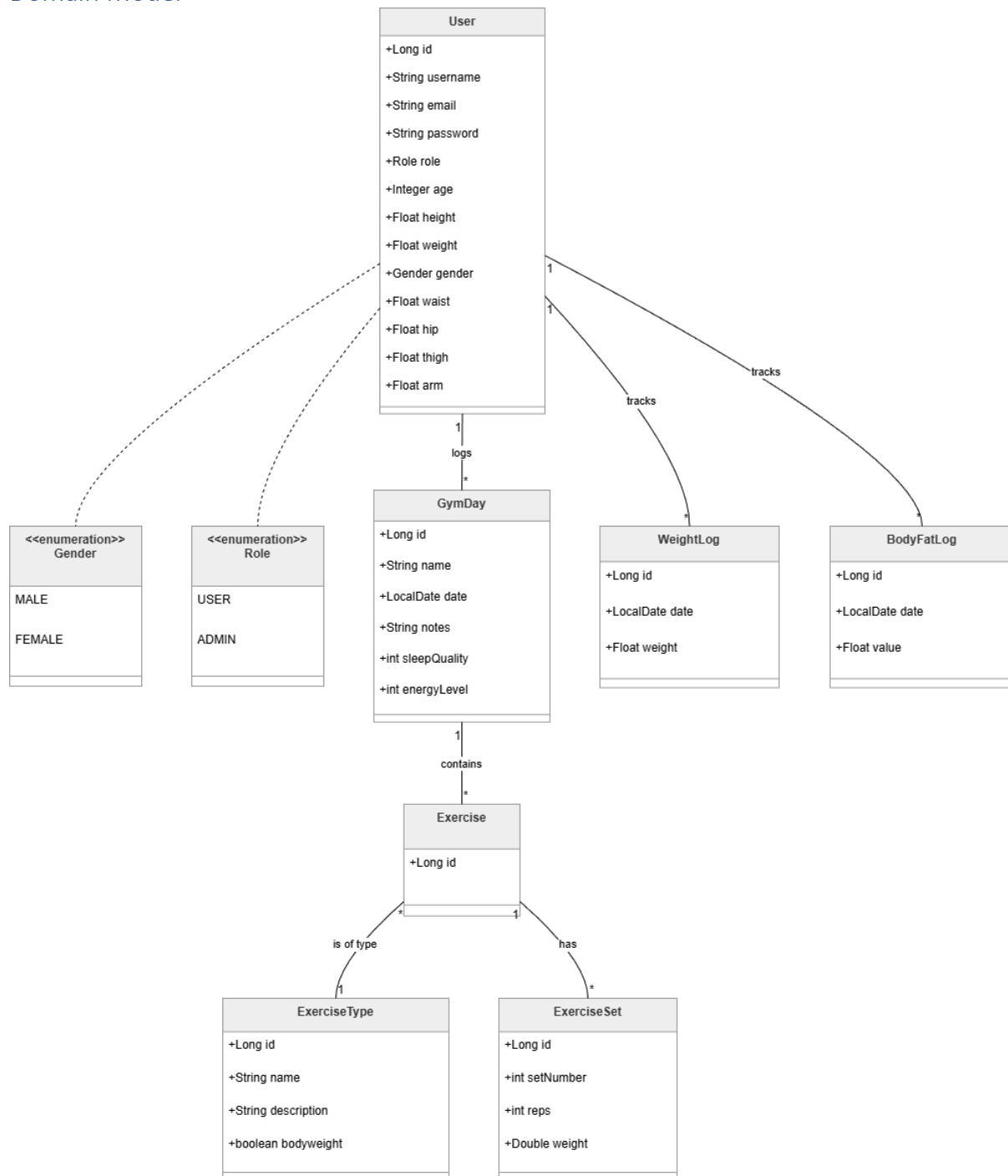
1. **Software Languages & Frameworks:** The Backend is based on Spring Boot Framework and will be written in Java. The Frontend client must be developed using React and Typescript.
2. **Database Management System:** The system must use PostgreSQL.
3. **Architectural Constraints:** The system must be designed as a distributed Client-Server application with a strict separation between the React frontend and Spring Boot backend. Communication between the two will be implemented via a RESTful API over HTTPS. The backend must use a Layered Architecture (most likely MVP).
4. **Development tools:** The project will utilize standard ecosystem tools, including Maven for Java dependency management, npm for React package management, Git for version control, VSCode and IntelliJ Idea for development, WSL2 for Linux support on Windows.

Glossary

1. **Gym Day** – a single calendar entry representing a workout session, which can contain multiple logged exercises and personal notes.
2. **Exercise Type** – a predefined template in a catalog, managed by an administrator that users select to log their activity.
3. **Standard User** – a role that allows individuals to manage their own personal workout logs and body metrics without access to other users' data.
4. **Administrator** – a role with elevated permission to manage the global exercise catalog.

Deliverable 2

Domain Model



Architectural Design

Conceptual Architecture

The system employs a Client Server Architecture using the layered pattern. The client is a standalone React Application and the server is composed of two independent Spring Boot microservices – one for login and one for all the other services.

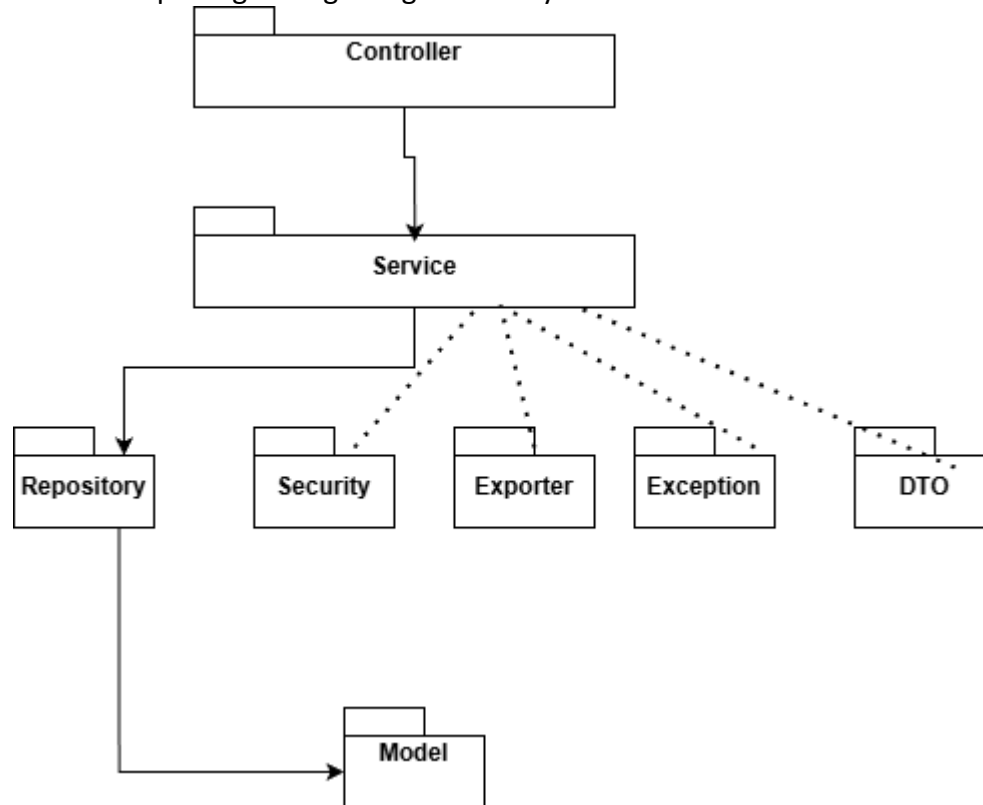
The frontend and backend are completely decoupled, they communicate exclusively via a RESTful API over HTTP. An Axios interceptor on the frontend routes requests to either Auth-Service(on port 8082) or the Gym-Service (Port 8080).

The frontend and backend communicate via RESTful API over HTTP, with the frontend sending the user's ID in the headers for identification.

The backend services follow the *Layered Architecture* pattern, using a Controller-Service-Repository pattern.

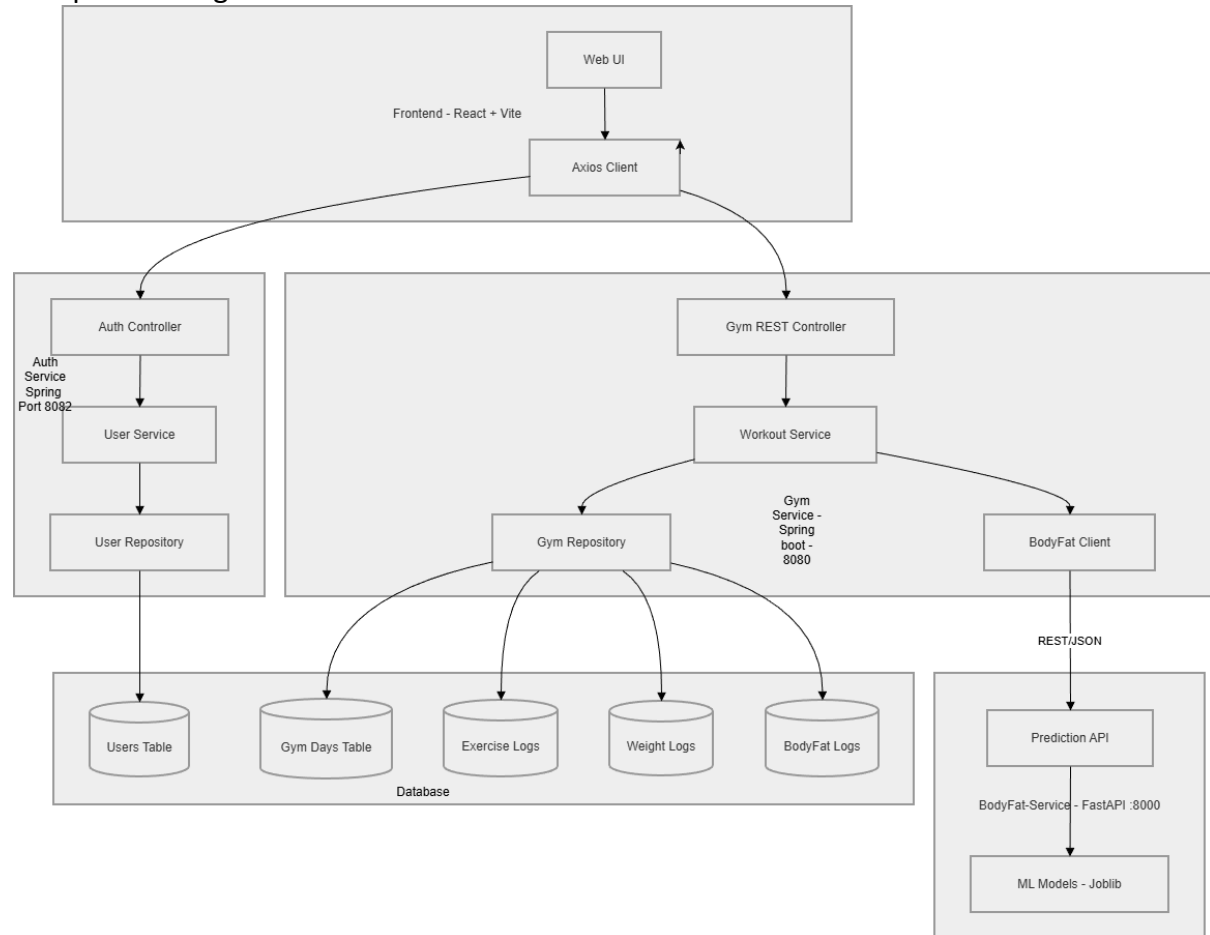
Package Design

The backend follows a strict layered structure, closely following SOLID principles. Below is shown the package design diagram for Gym-Service:

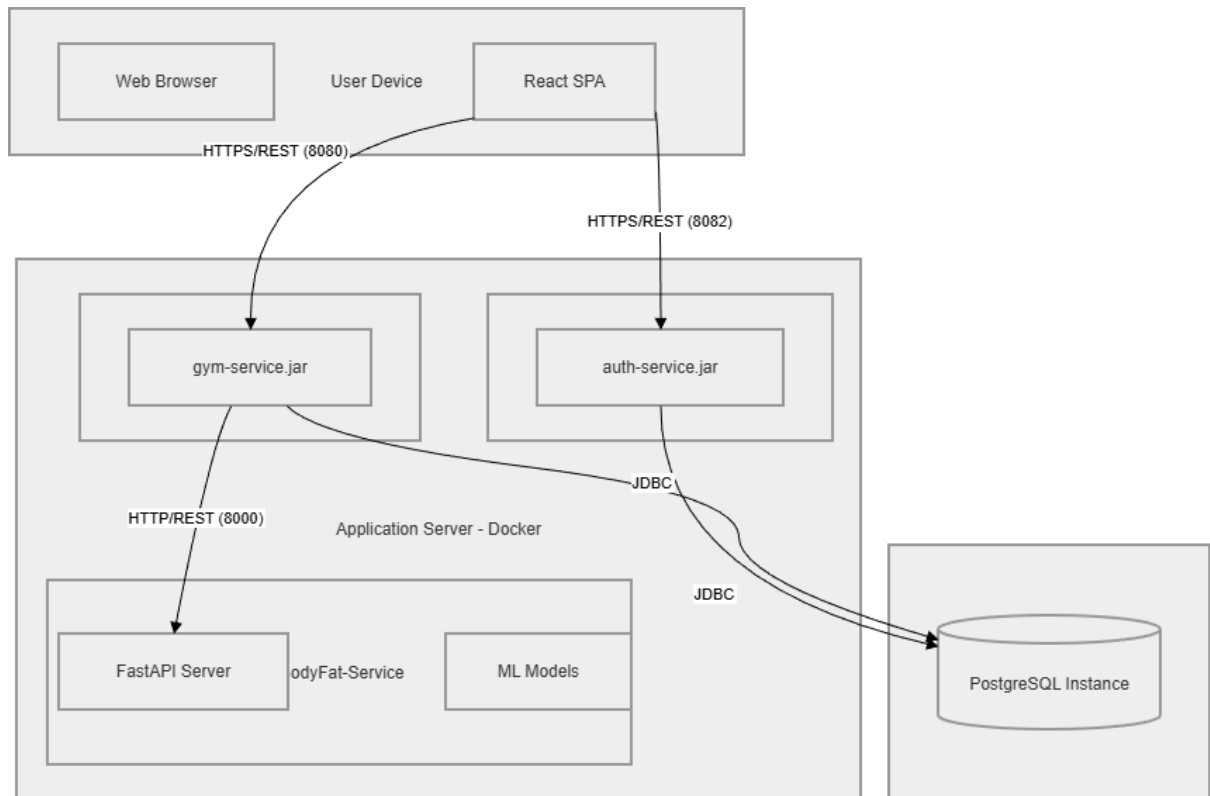


Component and Deployment Diagram

Component diagram:



Deployment Diagram:

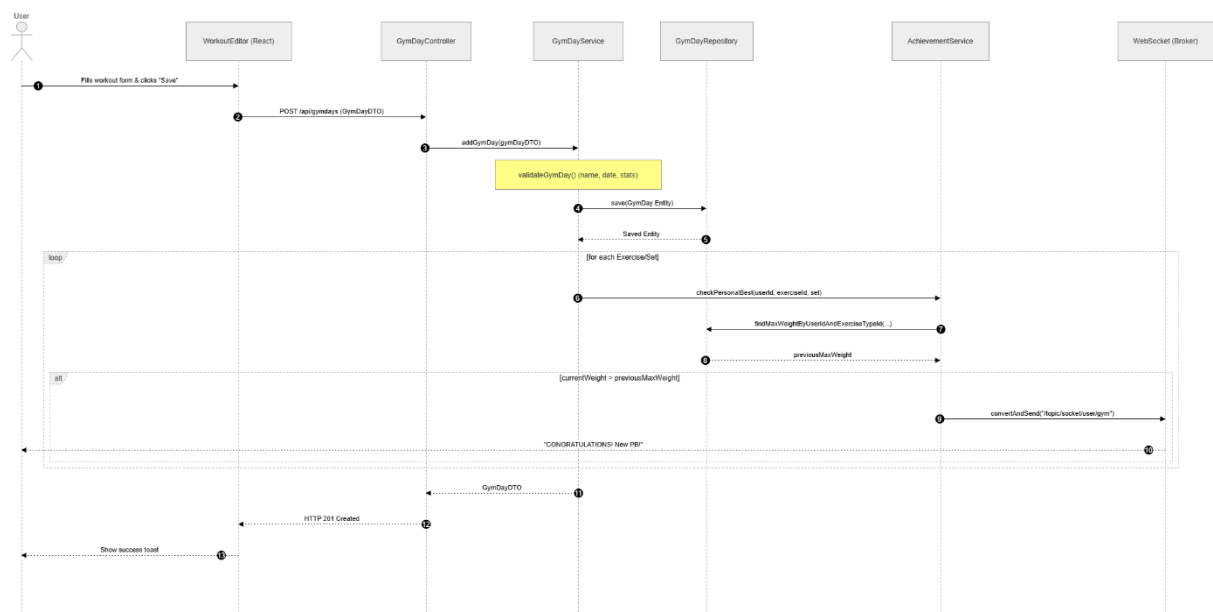


Deliverable 3

Design Model

Dynamic Behavior

Sequence Diagram for adding a workout and receiving an achievement

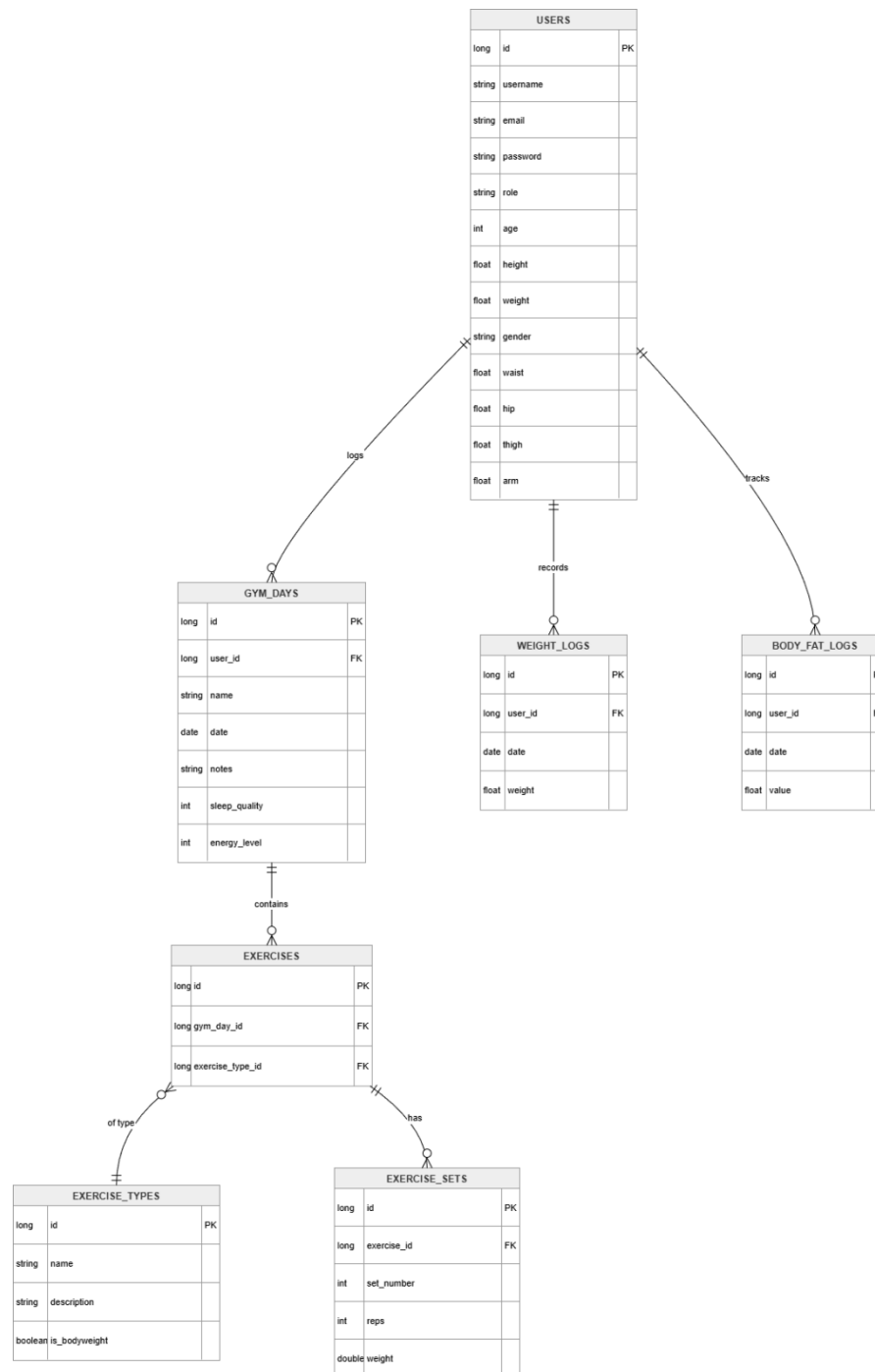


(A better image can be found in the repository, in the "doc" folder)

Sequence diagram for User Registration

4. Repository Pattern – Provides a collection-like interface for accessing domain objects, hiding the complexities of SQL and JPA.

Data Model



System Testing

The system was validated using unit testing and User Acceptance Testing

1. Unit Testing – using Junit.

Service layer validation was done this way

2. User Acceptance Testing – manual verification of user stories and use cases. Ensuring the UI reflects the state of the database.

Future Improvements

1. Interactive Progress Charts – Show trends in exercises at the gym, not just bodyweight, over days/months/years.

Conclusion

The GymBuddy application successfully transitions workout tracking from static spreadsheets to a dynamic, microservice-oriented ecosystem. By integrating a responsive React frontend with specialized Spring Boot and Python/FastAPI backends, the project achieves a robust separation of concerns and a highly maintainable architecture.

Key technical milestones include:

- * Architectural Integrity: The use of a decoupled microservices approach (Auth, Gym, and Body Fat services) ensures scalability and data isolation.
- * Design Pattern Implementation: The project leverages the Strategy Pattern for flexible XML/TXT report exporting, the Observer Pattern for real-time achievement notifications, and the Repository Pattern for streamlined database interactions, all of which adhere to the SOLID principles.
- * AI Integration: The successful implementation of a Python microservice for body fat prediction demonstrates the ability to bridge different technology stacks to provide advanced analytical features.

In summary, GymBuddy delivers a modern, high-performance solution for fitness enthusiasts, providing the tools necessary to quantify progress, optimize training through data-driven insights, and maintain a portable history of their athletic journey.

Bibliography